Cheaper Hindi Without Forgetting: Boundary-Aligned Self-Distillation for Tokenizer Extension

Aaryan Regmi
regmi_aaryan@srmap.edu.in

Yajnesh Ande
yajnesh_ande@srmap.edu.in

Sheldon Castelino
castelino_theodore@srmap.edu.in

SRM University–AP, Andhra Pradesh, India

November 2026

Abstract

Appending 256 Hindi merges to Qwen3-4B-Instruct-2507 cuts Hindi prompt tokens by 34%. Naive adaptation of the new tokens, however, breaks the model: Hindi reading falls and copying collapses (0/112 on a template probe). The copy failure traces to a handful of new tokens: the model emits a spaceless native character in place of a space-prefixed merged token, or drops a vowel sign. Continued-BPE merges nest inside native tokens, so every expanded-token boundary is also a native one, and the frozen native model is an exact teacher at those boundaries. We use this for *boundary-aligned self-distillation*: multi-layer hidden-state and next-token alignment at shared boundaries, plus training on the native model's own responses, with 256 new rows and rank-16 LoRA on 1.5M tokens. On a sealed TEST set (2,588 items, evaluated once) the distilled model keeps the full 34% token saving. Hindi Belebele comes within 1.9 points of the native model (95% CI $[-4.5, +0.6]$), English Belebele is exactly unchanged, and the model follows the answer format more reliably. Finding the answer span in Hindi QA (-5.0 points) and character-exact copying of varied text remain worse. An ablation shows the alignment terms matter: training on the teacher's responses alone leaves 24 of 112 template copies broken, and adding boundary alignment fixes all of them. We also show that strict benchmark scores overstate such recoveries, because they reward format and brevity, and we report a separate content score for every task.

1 Introduction

Hindi pays a large, tokenizer-specific token tax (Petrov et al., 2023; Ahia et al., 2023), and the tax turns into real compute under long-context, batched serving. On 1,012 aligned FLORES+ sentences (NLLB Team, 2022), Qwen2.5's tokenizer uses 4.49× [4.43, 4.54] as many tokens for Hindi as for English. A BLOOM tokenizer trained with more Indic text removes 92% of that excess, so the cost belongs to the tokenizer, not the language.

On short single prompts the latency penalty is only 1.00–1.03×. At 256-token contexts with batch size 4, prefill latency reaches 5.1× for Hindi and 7.5× for Tamil. KV-cache memory scales exactly with token count (Appendix A).

Extending a pretrained model's vocabulary is the direct fix, and on the tokenizer side it works. With 256 continued-BPE merges (Sennrich et al., 2016), Qwen3-4B-Instruct-2507 (Yang et al., 2025) needs 35.8% fewer tokens for held-out Hindi and 34.1% fewer for chat-rendered Belebele prompts, with English unchanged. The model side is the problem: after 1M tokens of standard adaptation, Hindi reading drops and literal copying fails.

Contributions.

1. **A token-level diagnosis of the failure (§3).** At 128 new tokens, the copy failure sits in 4 of the 22 new tokens the probe needs. At 4,096 new tokens the failure becomes romanization and digit collapse.
2. **Boundary-aligned self-distillation (§4).** The expanded tokenization is a strict coarsening of the native one, so the frozen native model is an exact teacher at every expanded boundary. No cross-tokenizer approximation is needed.
3. **A sealed-TEST evaluation that separates content from format (§5).** At the full token saving, Hindi reading comes within 1.9 points of native and English is unchanged. Extraction and character-exact copying remain measurably worse.

2 Setup

One pinned model, one ranked merge list, and paired comparisons against the unmodified model on the same items.

Model. `Qwen/Qwen3-4B-Instruct-2507`, revision `cdbee75f`, BF16, with the original weights frozen. New token IDs start after the full 151,936-row matrix, so every native ID, including chat special tokens, keeps its ID. Disabling the new rows and adapters reproduces native logits bit-identically on an H100.

Vocabulary. Continued BPE was fitted on a document-disjoint split of Sangraha Verified Hindi (Khan et al., 2024). The 128-, 256-, 512- and 4,096-token vocabularies are prefixes of one ranked merge list, and each passed one million round-trip checks. We use 256, the smallest size that clears a 25% prompt-token reduction on every Hindi task (Table 1).

Table 1: Token change against the native tokenizer, by vocabulary extension size.

New tokens	Hindi corpus	Belebele-hi prompts	Exact-detail prompts	English corpus
128	−30.0%	−28.4%	−22.8%	−0.08%
256	**−35.8%**	**−34.1%**	**−26.8%**	**−0.09%**
4,096	−45.9%	−44.5%	−41.2%	—

Evaluation. All tasks are scored on greedy generation from Qwen chat-rendered prompts. Token counts are measured on those rendered prompts.

- **DEV (822 items):** Belebele (Bandarkar et al., 2024) hin_Deva (176) and eng_Latn (184); XQuAD-hi (Artetxe et al., 2020) (192); the prior 112-item exact-detail copy probe; 8 mixed-script copy items; and a new 150-item diverse Hindi copy probe (one sentence each from 150 held-out documents, under 3 instruction templates).
- **TEST (2,588 items):** Belebele-hi (724), Belebele-en (716), XQuAD-hi (998), and a fresh 150-item copy probe from tokenizer-TEST documents.

Protocol. Benchmark membership was frozen by passage hash before any training. The candidate was selected on DEV and its checkpoint hash sealed before TEST was opened. Both the native control and the candidate were evaluated on TEST exactly once (Figure 1).

Statistics. Paired differences against the native model use 95% passage-clustered bootstrap intervals (10,000 resamples) and exact McNemar tests.

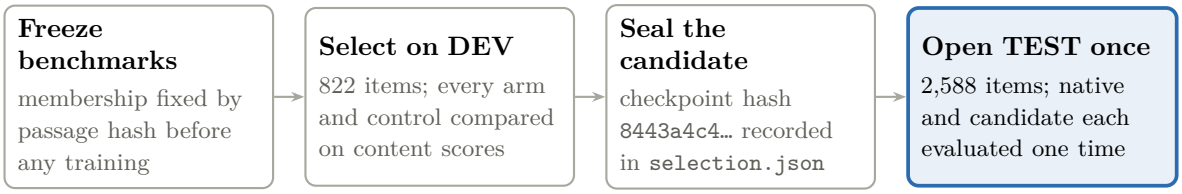


Figure 1: Evaluation protocol. The candidate was chosen on DEV and its checkpoint hash sealed before TEST was opened. The ablations in §5.5 ran on DEV only; TEST was never reused.

Scoring. Every task gets a strict score and a content score:

- **Belebele:** strict = the output is exactly the letter; content = the first answer letter.
- **XQuAD:** strict = exact match and token F1; content = whether the gold span appears in the output.
- **Copy probes:** strict = exact output; content = whether the reference appears in the output.

3 Why naive extension fails

The copy collapse after naive extension comes from a few specific new tokens, and its character changes with vocabulary size. All evidence comes from saved DEV predictions. We re-decoded every emitted ID sequence through the merge decomposition and the native tokenizer, and the result matches the stored text exactly. These are genuine model outputs, not detokenization artifacts.

The legacy copy probe is narrower than it looks. All 112 exact-detail references share one sentence template once digits are masked; only 39 references are distinct. The strict metric also penalizes the `[1]` label that the prompt invites: the native model scores 28/112 strict but contains the reference in 112/112. We therefore added the 150-item diverse probe and report containment alongside strict scores.

At 128 new tokens, four rows break copying. The template needs 22 distinct new tokens, and the model emits 18 of them at ≥90% of the required rate. The four failures are in Table 2.

Table 2: The four new tokens (of 22 needed) that the naive 128-row model fails to emit. ␣ marks a leading space.

New token	Native byte pieces	Needed	Emitted	Model emits instead
␣फ	Ġà¤ + «	112	3	फ, space lost
␣ज	Ġà¤ + Ł	336	96	ज, space lost
ैा	à¥ + Ī + à¥¤	112	32	ा, vowel sign lost
ींं	à¥Ç + à¤Ħ	224	115	ीं, nasal mark lost

Every item contains these tokens, so every item fails, even though mean character F1 is 0.988. Restricting decoding to native output IDs does not restore reading (101/176 either way), so the input side is implicated too.

At 4,096 new tokens the failure changes kind. The dominant errors become romanization (कविता ने → `kavita ne`, है। → `hai.`), digit collapse (२०२६ → २०००), and word substitution. Mean character F1 falls to 0.463. Small extensions fail at particular rows; large ones lose the script.

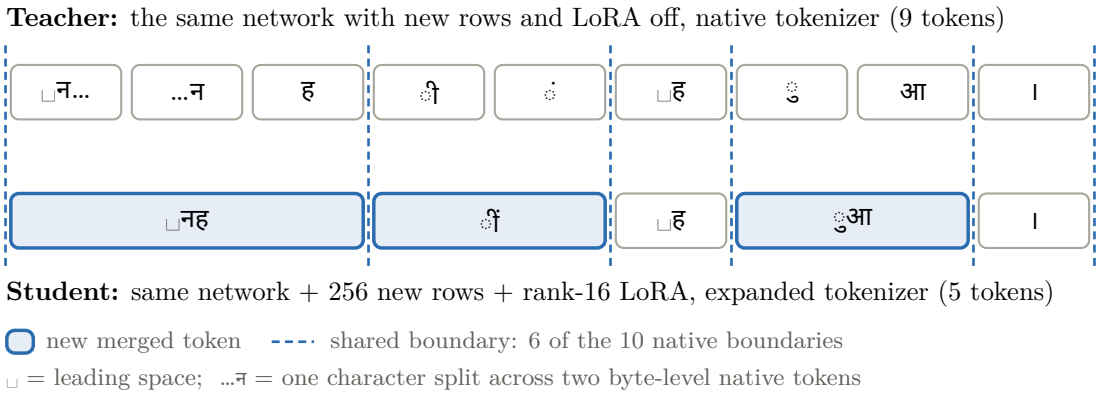


Figure 2: Boundary alignment on a real phrase, नहीं हुआ। ("did not happen"), under the native and 256-row tokenizers. The three new tokens each cover two or three native tokens, so all six expanded boundaries are native boundaries too. At those points the teacher's hidden states and next-token distribution are exact targets for the student.

Reading loss is real at 256 rows. Against native, the naive 256-row model loses 18 Belebele-hi items and gains 6 (strict, $p = 0.02$), and loses 31 and gains 10 on XQuAD span containment ($p = 0.001$).

4 Method: boundary-aligned self-distillation

The expanded model is trained to reproduce the frozen native model at every boundary the two tokenizations share. It also learns to emit the canonical merged tokens.

Exact alignment. Every new token decomposes into a fixed sequence of native tokens, and merges apply only inside pre-tokens. The expanded tokenization s of a text therefore expands exactly to its native tokenization n. Each student position j maps to the native position $e(j)$ where its last piece ends (Figure 2). We verified this on 400 chat-rendered training texts with no exceptions, special tokens included. The teacher is the same network with new rows and adapters switched off, which is bit-exact to the original. No second model copy and no cross-tokenizer marginalization are needed.

Data. No gold labels and no benchmark text are used.

- **Prompts:** 3,000 instruction prompts built from the adaptation split (2,400 Hindi, 600 English) across six tasks: summarize, list facts, extract the k-th sentence, translate, state the topic, and cloze multiple choice. Every template is worded differently from every evaluation template.
- **Raw text:** 1,000 segments.
- **Responses:** the native model answered every prompt greedily. Responses that hit the length limit, 53% of them, are kept as exact prefixes and trained without an end-of-turn token. Hindi's token cost is why so many hit the limit.

Loss. For each sequence, with $\hat{h}$ an L2-normalized hidden state, $L = \{9, 18, 27\}$ of 36 layers, W_0 the frozen native output head, and $\lambda_h = \lambda_k = 1$:

$$\begin{aligned}\mathcal{L} = \mathrm{CE}(s_{\text{resp}}) + \lambda_h \Big[\tfrac{1}{|L|} \sum_{l \in L} \big\|\hat{h}^S_{l,j} - \hat{h}^T_{l,e(j)}\big\|^2 + \big\|\hat{h}^S_{\text{final},j} - \hat{h}^T_{\text{final},e(j)}\big\|^2\Big] \\ + \lambda_k \,\mathrm{KL}\big(p_T(\cdot \mid e(j)) \,\big\|\, \mathrm{softmax}(W_0 \hat{h}^S_{\text{final},j})\big)\end{aligned} \tag{1}$$

The three terms do different jobs:

- **Cross-entropy** is computed on the teacher's responses (on all tokens for raw text), in the expanded tokenization with the full extended head. It is what teaches the canonical merged tokens such as ␣ज, the exact failure in §3.
- **The hidden-state term** averages over all shared boundaries, prompt included. That covers the input side, which §3 implicates in the reading loss.
- **The KL term** (Hinton et al., 2015) uses 128 random boundaries per sequence.

Parameters and budget. 256 untied new input and output rows, initialized as the mean of their native pieces, plus rank-16 LoRA (Hu et al., 2022) on all attention and MLP projections: 34.3M trainable parameters. Learning rates are 5×10^{-4} for the rows and 10^{-4} for LoRA (AdamW, accumulation 8). Training ran for one pass over 1.5M native-token-equivalents (2,061 sequences) in 44 minutes on a shared H100, peaking at 8.4 GiB. Native mode stays bit-exact because the adapters and rows can be switched off.

5 Results

On sealed TEST, the distilled model keeps the 34% Hindi token saving. It reads Hindi within 1.9 points of the native model and leaves English unchanged. Span extraction and character-exact copying remain worse.

5.1 Sealed TEST (evaluated once)

Table 3: Sealed TEST, content scores. Differences are ours minus native, in points, with 95% passage-clustered bootstrap CIs.

Task (items)	Content score	Native	Ours	Ours − native [95% CI]	McNemar p	Prompt tokens
Belebele-hi (724)	first answer letter	505	491	−1.9 [−4.5, +0.6]	0.18	−34.1%
Belebele-en (716)	first answer letter	640	640	0.0 [−0.6, +0.6]	1.0	0%
XQuAD-hi (998)	gold span in output	757	707	−5.0 [−7.5, −2.5]	<0.001	−35.2%
Diverse copy (150)	reference in output	121	92	−19.3 [−27.3, −11.3]	<0.001	−31.7%

Batch-16 and batch-1 greedy decoding agreed on 32/32 checked items for both models. The candidate checkpoint (`8443a4c4…`) matches the selection sealed on DEV.

Hindi reading is not significantly different from native, but the interval's lower end (−4.5) means non-inferiority within 2 points is not established. English is unchanged to within ±0.6 points.

5.2 Strict scores overstate recovery

On strict scores, the distilled model *beats* native on every Hindi task: Belebele-hi 491 vs. 459 (+4.4 [+1.4, +7.5]); XQuAD exact match 206 vs. 82 and token F1 0.477 vs. 0.353; diverse copy exact 90 vs. 84. These gains are format and brevity, not comprehension (Figure 3):

- **Belebele:** the distilled model answers with a bare letter on 724/724 Hindi items, against 650/724 for native, which sometimes appends the option text.
- **XQuAD:** its answers are shorter (DEV median 43 vs. 71 characters), which raises exact match and F1 even though the gold span appears less often.

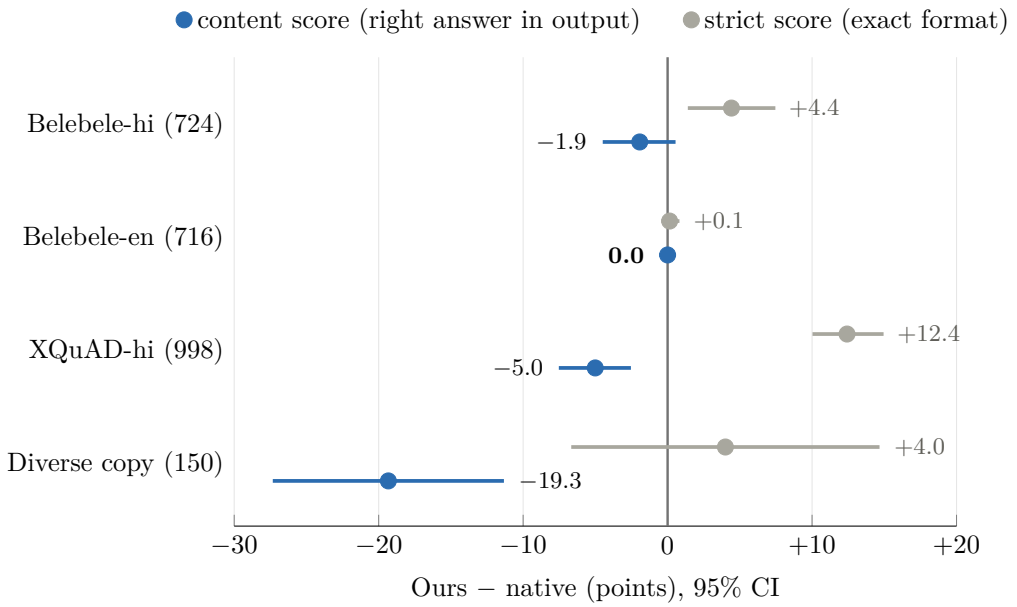


Figure 3: The same sealed-TEST predictions under two scorings. Every strict gain on Hindi turns into a loss once the answer is checked for content rather than format.

This matters beyond this paper. Vocabulary-adaptation results scored only by strict letter match or EM/F1 can report recovery that is really a change of output style.

5.3 Against the baselines (DEV)

Table 4: DEV, 822 items, content scores (items correct). Bold marks the notable failure of each baseline. [a]Trained on 1M tokens of raw text (80% Hindi, 20% English by bytes). Naive expansion trains only the 256 new rows plus a rank-64 adapter on them, with the model body frozen; native-tokenizer LoRA adds no tokens.

Model	Belebele -hi	Belebele -en	XQuAD-hi span	Exact-detail copy	Diverse copy	Hindi prompt tokens
Native (frozen)	119	173	137	112	121	baseline
Native-tokenizer LoRA[a]	122	**163**	129	111	138	0%
Naive 256-row expansion[a]	114	173	116	**0**	**34**	−34%
Ours (1.5M distillation)	118	173	125	112	89	−34%

Against naive expansion, distillation recovers about 80% of the Belebele-hi loss, 43% of the XQuAD loss and 63% of the diverse-copy loss, and it fully restores exact-detail copying (Table 4, Figure 4).

Ordinary adaptation with the native tokenizer is not a free alternative. It gains slightly on Hindi but costs English 5.4 points $[-8.8, -2.7]$. Distillation leaves English exactly unchanged on DEV and TEST, because the adapter learns the identity wherever the two tokenizations coincide.

5.4 What is still lost

The remaining copy errors are normalizations of the source, not the collapse of §3. On TEST, 20 of the 58 misses are a single change: a sentence-final "." written as the Hindi danda ।. Most of the rest are spelling standardizations (शुरु → शुरू), agreement changes (थे → था), and a few dropped vowel signs (कहा → कह).

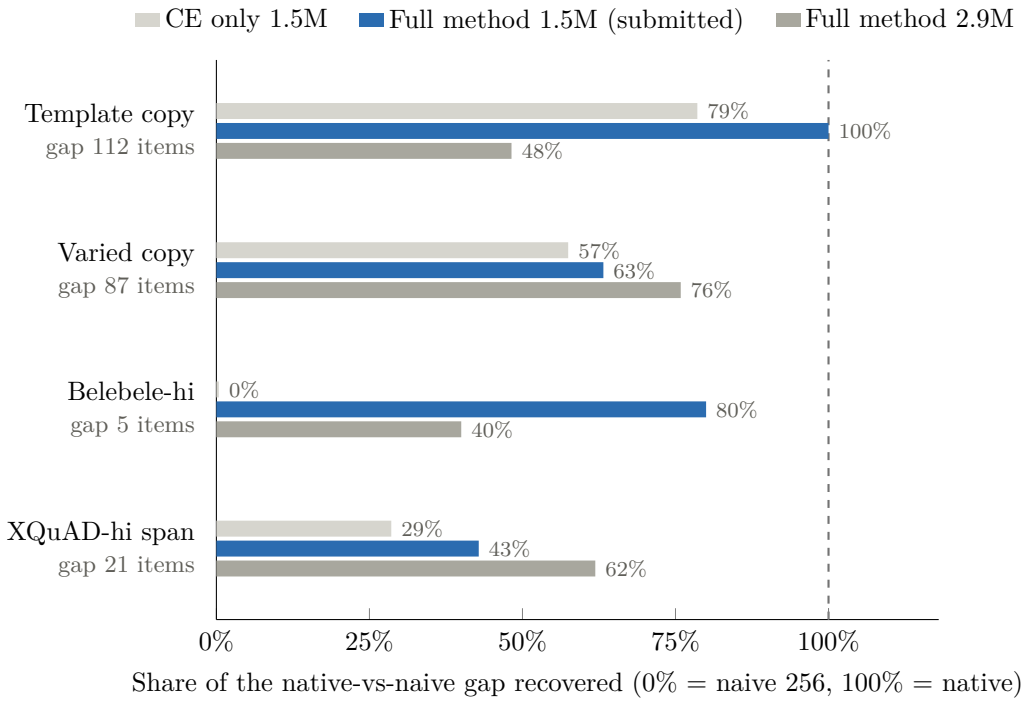


Figure 4: How much of the naive-extension loss each variant recovers on DEV (content scores); the two ablations are described in §5.5. Only the submitted model fully recovers template copying, and no variant fully recovers QA or varied copying. Belebele-hi's gap is only 5 items, so its percentages are noisy.

Mean character F1 is 0.995 against native's 0.851. The distilled model reproduces the content closely, but does not preserve nonstandard orthography. That rules it out, for now, where exact strings matter: IDs, names, legal text.

5.5 Ablations (DEV)

Is boundary alignment needed? We trained the same model on the teacher's responses with cross-entropy only (no hidden-state or KL terms), at the same 1.5M-token budget and seed. Table 5 pairs it with the full method on DEV, on content scores. "Wins" counts items only that model got right.

Table 5: Cross-entropy only vs. the full method (DEV, content scores).

Task (items)	Native	Naive 256	CE only	Full method	Wins full / CE	p (McNemar)
Exact-detail copy, contained (112)	112	0	88	**112**	24 / 0	<0.001
Diverse copy, contained (150)	121	34	84	89	15 / 10	0.42
Belebele-hi, first letter (176)	119	114	114	118	12 / 8	0.50
XQuAD span contained (192)	137	116	122	125	16 / 13	0.71
Belebele-en, first letter (184)	173	173	172	173	1 / 0	1.00

- **Alignment fixes the copy failure.** Cross-entropy alone leaves 24 of 112 template copies broken. The full method gets all 24 right and loses none. This is the failure the diagnosis traced to a few new tokens. Matching the native model's states at shared boundaries removes it; imitating its outputs does not.
- **Elsewhere the alignment terms help a little, but not significantly.** On Hindi

reading, cross-entropy alone scores the same as naive expansion (114), and the full method recovers to 118. With 176 items this is suggestive, not significant. Diverse copy and XQuAD move in the same direction, within noise.

- **Strict scores mislead again.** Cross-entropy alone scores higher on strict XQuAD exact match (53 vs. 38 of 192) because its answers are shorter, while it contains the answer less often (122 vs. 125).

Does more data close the copy gap? We ran the same method over all 4,000 adaptation prompts: 2.9M native tokens, about twice the main run (Table 6).

Table 6: Main run (1.5M tokens) vs. twice the data (2.9M tokens), DEV content scores.

Task (items)	Native	1.5M (main)	2.9M	Wins 1.5M / 2.9M	p (McNemar)
Template copy, contained (112)	112	**112**	54	58 / 0	<0.001
Varied copy, contained (150)	121	89	100	17 / 28	0.14
Belebele-hi, first letter (176)	119	118	116	11 / 9	0.82
XQuAD span contained (192)	137	125	129	10 / 14	0.54
Belebele-en, first letter (184)	173	173	173	0 / 0	1.00

- **More data moves the copy gap rather than closing it.** Varied copying and Hindi QA improve (100 vs. 89 and 129 vs. 125), though not significantly.
- **Template copying breaks on a single substitution.** All 58 new misses are the same one-character edit: the sentence-final verb है (“is”) becomes the plural हैं. Undoing that substitution makes all 58 correct. Every template item ends in है, so one learned habit fails most of the probe. Like the . → । habit above, it is a fluent-Hindi “correction” the model applies while copying.
- **Implication.** Character-exact copying needs a targeted objective, such as copy tasks in the adaptation mix or a copy-specific loss; more of the same teacher data does not provide it.

6 Related work

Vocabulary extension for Hindi is well studied. Our contribution is the exact-teacher property of continued BPE and a sealed, content-scored evaluation at a small budget.

Initialization. Joshi et al. (2026) study Hindi vocabulary extension and recommend subword-mean input rows, Hindi norm calibration and length-weighted output rows. We use subword-mean, untied rows; their initializers are complementary.

Staged expansion with large-scale training. Smith et al. (2026) train new rows, then the full model, on hundreds of billions of tokens. We use 1.5M tokens and keep the original weights frozen.

Token alignment and transfer. TokAlign (Li et al., 2025a), TokAlign++ (Li et al., 2026), VocADT (Han et al., 2025) and ZeTT (Minixhofer et al., 2024) map or learn embeddings for a new vocabulary.

Dynamic tokenization. Feher et al. (2025) retrofit models with dynamic tokenization and report mainly prefill and scoring results.

Cross-tokenizer distillation. Approximate likelihood matching (Minixhofer et al., 2025), byte-level distillation (Singh et al., 2026) and byte-prefix marginalization (Wang et al., 2026) handle teacher and student vocabularies that do not nest. Our student's tokenization is a strict coarsening of the teacher's. Alignment is exact at every student boundary, so hidden states can be matched layer by layer with no marginalization.

Instruction preservation. DIPLomA (Sarasua et al., 2025) and Li et al. (2025b) show that continued pretraining can erode instruction following. Our native-tokenizer LoRA control shows the same effect, an English loss, which self-distillation avoids.

None of these baselines was reproduced here. A head-to-head comparison is the most important missing experiment (§7).

7 Limitations and next steps

This is a single-model, single-seed, small-budget result; it supports a workshop paper, not yet a main-conference claim.

Limitations.

- **One model, one seed.** Qwen3-4B-Instruct-2507, seed 17. Seed variance is unmeasured.
- **Hindi reading parity is not established.** The TEST interval $[-4.5, +0.6]$ includes losses larger than our 2-point target.
- **Two content losses are significant.** XQuAD span containment (-5.0) and copy containment (-19.3).
- **No measured latency.** The token saving is measured; end-to-end latency on an uncontended GPU is not. Our earlier measurements suggest gains appear mainly in long-context, batched serving.
- **Provenance.** The adaptation data (Sangraha Verified) has document IDs but no website URLs, so the train/test split is document-disjoint but not provably website-disjoint.
- **Teacher data.** 53% of teacher responses were truncated at 64–160 tokens and trained as prefixes.
- **Selection.** The planned 3M-token candidate could not be trained before TEST because the GPU was shared. The 1.5M model was the only eligible candidate; this amendment is recorded in the selection file.

Next steps, in priority order.

1. **Three seeds** of the main arm and both ablations on DEV.
2. **Copy fidelity.** Add verbatim-copy items to the distillation mix, preserving source punctuation and spelling, and measure whether the danda normalization (20 of 58 TEST copy misses) goes away.
3. **Reproduce one published baseline at matched budget:** the Joshi et al. (2026) initializer, or TokAlign++ alignment.
4. **A second model family**, for example Mistral-7B-Instruct-v0.3, which is pinned in the protocol.
5. **Latency:** batch-1 and batch-8 complete-response timing on an uncontended GPU, at short and long contexts.
6. **Hindi instruction following:** IndicIFEval, whose official scorer is already integrated.

References

Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David Mortensen, Noah Smith, and Yulia Tsvetkov. Do all languages cost the same? tokenization in the era of commercial language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

Mikel Artetxe, Sebastian Ruder, and Dani Yogatama. On the cross-lingual transferability of monolingual representations. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.

Lucas Bandarkar, Davis Liang, Benjamin Muller, Mikel Artetxe, Satya Narayan Shukla, Donald Husa, Naman Goyal, Abhinandan Krishnan, Luke Zettlemoyer, and Madian Khabsa. The Belebele benchmark: a parallel reading comprehension dataset in 122 language variants. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.

Darius Feher, Ivan Vulić, and Benjamin Minixhofer. Retrofitting large language models with dynamic tokenization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 29866–29883. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1444. URL https://aclanthology.org/2025.acl-long.1444/.

HyoJung Han, Akiko Eriguchi, Haoran Xu, Hieu Hoang, Marine Carpuat, and Huda Khayrallah. Adapters for altering LLM vocabularies: What languages benefit the most? In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2410.09644.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Raviraj Joshi, Utkarsh Vaidya, Sanjay Singh Chauhan, and Niranjan Wartikar. Beyond initialization loss: A systematic study of token embedding initialization strategies for LLM vocabulary extension. *arXiv preprint arXiv:2608.03494*, 2026. URL https://arxiv.org/abs/2608.03494.

Mohammed Safi Ur Rahman Khan et al. IndicLLMSuite: A blueprint for creating pre-training and fine-tuning datasets for Indian languages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.

Chong Li, Jiajun Zhang, and Chengqing Zong. TokAlign: Efficient vocabulary adaptation via token alignment. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4109–4126. Association for Computational Linguistics, 2025a. doi: 10.18653/v1/2025.acl-long.207. URL https://aclanthology.org/2025.acl-long.207/.

Chong Li, Yingzhuo Deng, Wen Yang, Jiajun Zhang, and Chengqing Zong. TokAlign++: Advancing vocabulary adaptation via better token alignment. *arXiv preprint arXiv:2605.13429*, 2026. URL https://arxiv.org/abs/2605.13429.

Zihao Li, Shaoxiong Ji, Hengyu Luo, and Jörg Tiedemann. Rethinking multilingual continual pretraining: Data mixing for adapting LLMs across languages and resources. *arXiv preprint arXiv:2504.04152*, 2025b. URL https://arxiv.org/abs/2504.04152.

Benjamin Minixhofer, Edoardo Maria Ponti, and Ivan Vulić. Zero-shot tokenizer transfer. *arXiv preprint arXiv:2405.07883*, 2024. URL https://arxiv.org/abs/2405.07883.

Benjamin Minixhofer, Ivan Vulić, and Edoardo Maria Ponti. Universal cross-tokenizer distillation via approximate likelihood matching. *arXiv preprint arXiv:2503.20083*, 2025. URL https://arxiv.org/abs/2503.20083.

NLLB Team. No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*, 2022.

Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems*, 2023.

Ixak Sarasua, Ander Corral, and Xabier Saralegi. DIPLomA: Efficient adaptation of instructed LLMs to low-resource languages via post-training delta merging. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 24898–24912. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-emnlp.1355. URL https://aclanthology.org/2025.findings-emnlp.1355/.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2016.

Avyav Kumar Singh, Yen-Chen Wu, Alexandru Cioba, Alberto Bernacchia, and Davide Buffelli. Cross-tokenizer LLM distillation through a byte-level interface. *arXiv preprint arXiv:2604.07466*, 2026. URL https://arxiv.org/abs/2604.07466.

Jimmy T. H. Smith, Tarek Dakhran, Alberto Cabrera, Simon S. Lee, Paul Pak, Aditya Tadimeti, Tim Seyde, Maxime Labonne, Alexander Amini, and Mathias Lechner. In-place tokenizer expansion for pre-trained LLMs. *arXiv preprint arXiv:2607.15232*, 2026. URL https://arxiv.org/abs/2607.15232.

Hao Wang, Kun Yuan, Wenlin Zhong, Minglei Zhang, Han Xiao, Ming Sun, and Honggang Qi. Cross-tokenizer on-policy distillation via byte-prefix marginalization. *arXiv preprint arXiv:2607.22334*, 2026. URL https://arxiv.org/abs/2607.22334.

An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL https://arxiv.org/abs/2505.09388.

A Token tax, compute, and why not KV compression

These results come from the earlier ELIP experiments on Qwen2.5-0.5B-Instruct and BLOOM-560m. Every headline number was recomputed from the raw result files.

Token inflation. Per-example ratio of tokens(L) to tokens(en) on FLORES+ devtest, 1,012 sentences per language, with 95% bootstrap CIs (Table 7). Every Hindi and Tamil sentence costs more tokens than its English pair under Qwen. BLOOM removes 92% of Hindi's excess but is worse for Thai.

Table 7: Token inflation relative to English, FLORES+ devtest.

Language	Qwen2.5	Mistral-7B-v0.1	BLOOM-560m
Tamil	6.23 [6.16, 6.30]	5.97	1.29
Hindi	4.49 [4.43, 4.54]	4.64	1.29
Thai	2.58	4.31	4.62

Tokens versus compute. CUDA-event timing, ratio of Hindi or Tamil latency to English:

- **Short prompts, batch 1:** 1.00–1.03×.
- **256-token contexts, batch 4:** 5.1× (Hindi) and 7.5× (Tamil) prefill latency.
- **KV-cache bytes:** exactly linear in tokens.
- **Last-position logits only:** the ratios do not change.

Why we did not compress after tokenization. We tried several ways of compressing hidden states inside the model:

- **Strided compression at layer 11 (1.5×)** cut next-token KL by 87% but saved only 16.6% of KV, and retrieval exact match fell from 0.66 to 0.01. Low KL did not predict task behaviour.
- **Oracle versus learned selectors.** A gold-evidence oracle retained 0.61–0.69 of quality. Every deployable selector retained only 0.15–0.17, across four fresh test sets, even with 100% evidence recall.
- **Learned compressors and adaptation.** Learned compressors never beat fixed striding, and LoRA adaptation did not repair the damage.

The cost is better removed at the tokenizer, which motivates this paper.

B Reproducibility

Code and data are at https://github.com/Aaryan73/vocab-distill.

- **Code:** `vocab_distill/` (tokenizer extension, model with new rows, losses, decoding, scoring) and one script per step: `fit_tokenizer.py`, `make_prompts.py`, `teacher.py`, `train.py` (methods `distill`, `ce-only`, `naive`, `native-lora`), `evaluate.py` and `analyze.py`.
- **Data:** the 256 merges, the teacher responses, the raw text the baselines trained on, and the DEV and TEST suites. Rebuilding the tokenizer, prompts and copy probes from the source splits reproduces the shipped files exactly.
- **Results:** every DEV and TEST prediction. `analyze.py` recomputes every table in this paper from them without a GPU.
- **Hashes:** main checkpoint `8443a4c4c8e3c81c...`; TEST suite `4c6a6ff22d501423...`.